

Universidad Nacional Mayor de San Marcos
Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas



TRABAJO GRUPAL:

Gestión de Flujo de Aprobaciones - CAMPUS360

DOCENTE:

Andrade Mogollon, Teodoro Manuel

ASIGNATURA:

DISEÑO DE SISTEMAS DE INFORMACIÓN

INTEGRANTES:

Condori Pari, Arnol

Huaranca Quilca, Gustavo Enrique

Moya Laureano, Jussel Enrique

Ninamango Mucha, Demian Joseph

Quispe Paricahua,, Dante Jussie

Ticsihua De La Cruz, Erick Enerson

Lima - Perú

2026

ÍNDICE

1. Introducción y Justificación Arquitectónica	3
1.1. Objetivo de la aplicación de patrones en el Módulo de Aprobaciones.	3
1.2. Resumen de patrones seleccionados	3
2. Patrón Creacional: Factory Method	4
2.1. El Problema: La complejidad de instanciar solicitudes con diferentes reglas de SLA	4
2.2. La Solución: Implementación de SolicitudFactory	4
2.3. Evidencia en Código: Fragmento de solicitud_factory.py	4
2.4. Beneficio: Cumplimiento del Principio Abierto/Cerrado (OCP) de SOLID.	5
3. Patrón Estructural: Facade (Fachada)	5
3.1. El Problema: Acoplamiento y sobrecarga de responsabilidades en la API	5
3.2. La Solución: Implementación de BandejaAprobacionFacade	6
3.3. Evidencia en Código: Fragmento de bandeja_facade.py	6
3.4. Beneficio: Simplificación de interfaces y cumplimiento SRP	7
4. Patrón de Comportamiento: Strategy	7
4.1. El Problema: Alta complejidad ciclomática y código espagueti	7
4.2. La Solución: Implementación de la interfaz SlaStrategy y su Resolver	8
4.3. Evidencia en Código: Fragmento de sla_strategy.py	8
4.4. Beneficio: Eliminación de código espagueti y facilidad para agregar nuevos tipos de trámites en el futuro.	9
5. Conclusiones	10
5.1. Impacto de los patrones en la mantenibilidad y testabilidad del sistema	10

1. Introducción y Justificación Arquitectónica

1.1. Objetivo de la aplicación de patrones en el Módulo de Aprobaciones.

El desarrollo del Módulo de Aprobaciones para la plataforma Campus360 presentó desafíos arquitectónicos significativos, específicamente en la gestión del ciclo de vida de las solicitudes, el cálculo dinámico de Acuerdos de Nivel de Servicio (SLA) y la separación de responsabilidades entre la capa de presentación (API REST) y la lógica de negocio.

El objetivo principal de aplicar Patrones de Diseño en este módulo es estructurar el código fuente para garantizar una arquitectura robusta, escalable y mantenible. Su implementación nos ha permitido:

- **Cumplir con los principios SOLID:** En particular, el Principio de Responsabilidad Única (SRP) para evitar clases monolíticas y el Principio de Inversión de Dependencias (DIP) para desacoplar el dominio de la infraestructura tecnológica.
- **Facilitar la Testabilidad:** Al aislar los componentes y depender de abstracciones, se habilitó la inyección de Mocks y Stubs para la ejecución de pruebas unitarias exhaustivas sin depender de la base de datos.
- **Mitigar Code Smells:** La aplicación de patrones permitió reducir la complejidad ciclomática, eliminando condicionales anidados excesivos y erradicando la duplicación de código, alineándose con el principio DRY (Don't Repeat Yourself).

1.2. Resumen de patrones seleccionados

Para cumplir con los estándares de calidad del diseño de sistemas y resolver los problemas estructurales identificados durante el desarrollo del MVP, se han seleccionado e implementado tres patrones de diseño fundamentales:

1. **Patrón Creacional - Factory Method:** Implementado a través de la clase SolicitudFactory. Su propósito es encapsular la complejidad de instanciación de las entidades de dominio (Solicitudes) y delegar la inyección de sus respectivas estrategias de SLA, protegiendo al sistema de modificaciones en cascada cuando se introduzcan nuevos tipos de trámites.
2. **Patrón Estructural - Facade (Fachada):** Materializado en la clase BandejaAprobacionFacade. Su propósito es simplificar la comunicación entre los Controladores (Web API) y la capa de Infraestructura, proveyendo una interfaz unificada que orquesta el mapeo de DTOs, el cálculo de semáforos de SLA y el ordenamiento complejo, ocultando esta lógica subyacente al cliente.
3. **Patrón de Comportamiento - Strategy:** Implementado mediante la interfaz SlaStrategy y sus clases concretas (TramiteRegularStrategy, TramiteUrgenteStrategy). Su propósito es extraer y encapsular los diferentes algoritmos de cálculo de tiempos límite y prioridad de atención, permitiendo que el sistema asigne el comportamiento adecuado dinámicamente en tiempo de ejecución sin alterar el flujo principal.

2. Patrón Creacional: Factory Method

2.1. El Problema: La complejidad de instanciar solicitudes con diferentes reglas de SLA

Durante las iteraciones iniciales del Módulo de Aprobaciones, la creación de una nueva entidad **SOLICITUD** presentaba un alto grado de acoplamiento y violaba el Principio de Responsabilidad Única (SRP). El Repositorio (o el Controlador) se veía obligado a utilizar múltiples bloques de sentencias condicionales (if/else) para determinar la lógica de negocio aplicable según el tipo de trámite (por ejemplo, si era "Regular" o "Urgente").

Esta estructura generaba un "Code Smell" conocido como **Long Method** y obligaba a la capa de persistencia a calcular fechas de Acuerdo de Nivel de Servicio (SLA) y prioridades, mezclando lógica de negocio con lógica de acceso a datos. Además, cualquier cambio en los tiempos de respuesta exigía modificar el código core, arriesgando la estabilidad del sistema.

2.2. La Solución: Implementación de SolicitudFactory

Para resolver esta deficiencia estructural, se implementó el patrón de diseño creacional Factory Method . Se diseñó la clase SolicitudFactory, cuya única responsabilidad es centralizar y encapsular la lógica de creación del Aggregate Root (la entidad Solicitud).

En lugar de instanciar la entidad directamente en el controlador o repositorio, la fábrica recibe los parámetros específicos del trámite (tipo_tramite, solicitante, estado_inicial_id) junto con una abstracción del comportamiento (SlaStrategy). La fábrica inyecta esta estrategia resuelta para calcular los tiempos y la prioridad, asegurando que la entidad nazca íntegra y con las reglas de negocio aplicadas antes de tocar la base de datos.

2.3. Evidencia en Código: Fragmento de solicitud_factory.py

```
"""
Capa de Servicios: Lógica Creacional (Factory Pattern).
Desacopla la instanciación compleja de la entidad principal
(Solicitud),
centralizando su ensamblaje y asegurando el cumplimiento
estricto de SOLID.
"""
from app.domain.models import Solicitud
from app.services.sla_strategy import SlaStrategy

class SolicitudFactory:
    """
    Patrón Creacional Factory: Ensambla la entidad
    'Solicitud' integrando
    las reglas del negocio antes de su persistencia en la
    base de datos.
    """
```

```

"""

    @staticmethod
    def crear_solicitud(tipo_tramite: str, solicitante: str,
estado_inicial_id: int, estrategia: SlaStrategy) ->
Solicitud:
    """
    Construye el Aggregate Root inyectando la estrategia
de SLA resuelta (DIP).
    Asegura que la entidad nazca con su prioridad y
tiempos correctamente calculados.
    """
    nueva_solicitud = Solicitud(
        tipoSolicitud=tipo_tramite,
        solicitante=solicitante,
        estado_id=estado_inicial_id,
        slaObjetivo=estrategia.calcular_sla(),
        prioridad=estrategia.obtener_prioridad(),
        adjuntos=[]
    )
    return nueva_solicitud

```

2.4. Beneficio: Cumplimiento del Principio Abierto/Cerrado (OCP) de SOLID.

El principal beneficio arquitectónico de esta refactorización es el estricto cumplimiento del Principio Abierto/Cerrado (OCP) y el Principio de Inversión de Dependencias (DIP) de SOLID

Actualmente, el sistema está cerrado para la modificación: el código de SolicitudFactory no necesitará ser alterado si las reglas del negocio cambian. Al mismo tiempo, está abierto para la extensión: si la universidad requiere un nuevo flujo, solo se deberá crear una nueva clase SlaStrategy y pasarla como parámetro. Esto garantiza una mantenibilidad a largo plazo.

3. Patrón Estructural: Facade (Fachada)

3.1. El Problema: Acoplamiento y sobrecarga de responsabilidades en la API

En el diseño inicial del caso de uso "Listar Bandeja de Pendientes", el Controlador de la API estaba fuertemente acoplado a la lógica de negocio. Para construir la vista que requiere el Frontend, el controlador no solo debía consultar directamente a la base de datos, sino que también era responsable de iterar sobre los registros, calcular dinámicamente el estado del SLA (asignando el semáforo visual de urgencia) y ejecutar algoritmos de ordenamiento multicriterio.

Esta arquitectura generaba controladores excesivamente grandes ("Fat Controllers"), violando el Principio de Responsabilidad Única (SRP) y dificultando enormemente la creación de pruebas unitarias y el mantenimiento del código

3.2. La Solución: Implementación de BandejaAprobacionFacade

Para mitigar esta deuda técnica y cumplir con las directrices arquitectónicas de separación estricta en capas, se implementó el patrón de diseño estructural Facade (Fachada).

Se creó la clase **BandejaAprobacionFacade** en la capa de Aplicación/Servicios. Esta fachada actúa como un orquestador inteligente que oculta toda la complejidad del subsistema: interactúa con el **solicitud_repository** para obtener las entidades ORM crudas, procesa el cálculo dinámico del semáforo temporal (SLA vencido vs. a tiempo), y finalmente aplica las reglas de negocio para ordenar y mapear los resultados a Objetos de Transferencia de Datos (**SolicitudDTO**) limpios y estandarizados, listos para ser consumidos por el cliente web.

3.3. Evidencia en Código: Fragmento de bandeja_facade.py

```
"""
Capa de Servicios: Lógica de Orquestación.
Aplica el Patrón Estructural Facade para encapsular reglas de
negocio complejas,
liberando a los controladores (Capa API) de responsabilidades
de transformación de datos (SRP).
"""
from sqlalchemy.orm import Session
from datetime import datetime
from app.repositories import solicitud_repository
from app.domain.schemas import SolicitudDTO

class BandejaAprobacionFacade:
    """
    Patrón Facade: Oculta la complejidad de interactuar con
    repositorios,
    mapear entidades ORM a DTOs y calcular métricas dinámicas
    operativas (SLA).
    """
    def __init__(self, db: Session):
        self.db = db

    def obtener_bandeja_ordenada(self) -> list[SolicitudDTO]:
        """
        Orquesta la construcción de la bandeja. Aplica reglas
        de ordenamiento
        del negocio: Prioridad estricta y control de
        vencimiento de SLA (Semáforo).
        """
        # 1. Obtener datos crudos de la capa de persistencia
        solicitudes_db =
solicitud_repository.listar_solicitudes_por_aprobar(self.db)

        # 2. Mapear a DTOs y calcular Semáforo Dinámico en
```

```

tiempo de ejecución
    dto_list = []
    for sol in solicitudes_db:
        semaforo = "ROJO" if sol.slaObjetivo <
datetime.now() else "VERDE"

        dto = SolicitudDTO(
            id=sol.idSolicitud,
            alumno=sol.solicitante,
            tipo_tramite=sol.tipoSolicitud,
            estado=sol.estado_actual.tipoEstado,
            prioridad=sol.prioridad,
            semaforo_sla=semaforo
        )
        dto_list.append(dto)

    # 3. Lógica de Ordenamiento del Negocio (Primero
    Urgentes, luego SLA)
    priority_order = {"ALTA": 1, "NORMAL": 2, "BAJA": 3}

    # Ordenamos: Primero por prioridad jerárquica, luego
    por riesgo de SLA (Rojo primero)
    dto_list.sort(key=lambda x:
(priority_order.get(x.prioridad, 99), x.semaforo_sla ==
"VERDE"))

    return dto_list

```

3.4. Beneficio: Simplificación de interfaces y cumplimiento SRP

El uso del patrón Facade aporta dos beneficios estructurales definitivos al Módulo de Aprobaciones. Primero, garantiza el estricto cumplimiento del Principio de Responsabilidad Única (SRP), ya que el controlador de la API ahora se limita exclusivamente a enrutar peticiones HTTP y retornar la lista JSON que la fachada le provee, ignorando los detalles de su construcción.

Segundo, simplifica radicalmente la integración con el cliente frontend: la complejidad del ensamblaje, la evaluación algorítmica de tiempos y el ordenamiento quedan encapsulados en el backend, evitando que la lógica de dominio se filtre hacia la interfaz de usuario, y favoreciendo un diseño de alta cohesión y bajo acoplamiento.

4. Patrón de Comportamiento: Strategy

4.1. El Problema: Alta complejidad ciclomática y código espagueti

Durante la fase de análisis y primera implementación del ciclo de vida de las solicitudes, la lógica para determinar el Acuerdo de Nivel de Servicio (SLA) y la prioridad operativa se encontraba incrustada dentro de los servicios principales. Se utilizaban múltiples sentencias

condicionales anidadas (if/elif/else) para evaluar el tipo de trámite y, en base a ello, calcular las horas de resolución.

Este enfoque generaba "Code Smells" evidentes como Long Method y falta de cohesión . A medida que el catálogo de servicios de Campus360 crecía, la cadena de condicionales se volvía insostenible, aumentando la complejidad ciclomática del código, violando el Principio de Responsabilidad Única (SRP) y haciendo que el sistema fuera extremadamente frágil ante cambios

4.2. La Solución: Implementación de la interfaz SlaStrategy y su Resolver

Para erradicar este problema y cumplir con el requisito de implementar un patrón de comportamiento , se optó por el patrón Strategy. Este patrón permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables.

Se extrajo toda la lógica de cálculo temporal y de priorización hacia una abstracción principal: la interfaz **SlaStrategy**. A partir de ella, se implementaron clases concretas independientes, como **TramiteRegularStrategy** (72 horas) y **TramiteUrgenteStrategy** (24 horas). Adicionalmente, para potenciar el desacoplamiento, se implementó un **SlaStrategyResolver** que actúa como inyector de dependencias, evaluando el tipo de trámite y retornando la estrategia correcta instanciada.

4.3. Evidencia en Código: Fragmento de sla_strategy.py

```
"""
Capa de Servicios: Lógica de Comportamiento (Strategy
Pattern).
Aplica el Patrón Strategy para el cálculo dinámico de
Acuerdos de Nivel
de Servicio (SLA) y prioridades. Garantiza el principio
Abierto/Cerrado (OCP).
"""
from datetime import datetime, timedelta
from abc import ABC, abstractmethod

class SlaStrategy(ABC):
    """
    Contrato base (Interfaz). Define la familia de algoritmos
    intercambiables para el cálculo de SLAs operativos.
    """
    @abstractmethod
    def calcular_sla(self) -> datetime:
        pass

    @abstractmethod
```

```

    def obtener_prioridad(self) -> str:
        pass

class TramiteRegularStrategy(SlaStrategy):
    """Estrategia Concreta: Trámites estándar con SLA de 72
    horas (Prioridad NORMAL)."""
    def calcular_sla(self) -> datetime:
        return datetime.now() + timedelta(hours=72)

    def obtener_prioridad(self) -> str:
        return "NORMAL"

class TramiteUrgenteStrategy(SlaStrategy):
    """Estrategia Concreta: Trámites críticos con SLA de 24
    horas (Prioridad ALTA)."""
    def calcular_sla(self) -> datetime:
        return datetime.now() + timedelta(hours=24)

    def obtener_prioridad(self) -> str:
        return "ALTA"

class SlaStrategyResolver:
    """
    Inyector de Dependencias Estático. Desacopla la selección
    de la estrategia
    del Factory, asegurando que la infraestructura cumpla con
    OCP y DIP.
    """
    @staticmethod
    def resolver(tipo_tramite: str) -> SlaStrategy:
        if "Extemporánea" in tipo_tramite or "Urgente" in
tipo_tramite:
            return TramiteUrgenteStrategy()
        return TramiteRegularStrategy()

```

4.4. Beneficio: Eliminación de código espagueti y facilidad para agregar nuevos tipos de trámites en el futuro.

La adopción del patrón Strategy transformó un código rígido en uno altamente dinámico. El principal beneficio es el estricto cumplimiento del Principio Abierto/Cerrado (OCP) y el Principio de Inversión de Dependencias (DIP) de SOLID .

Al eliminar el código espagueti de if/else, el sistema actual no necesita ser modificado en su núcleo si cambian las políticas de tiempos de la facultad. Si el próximo semestre el área administrativa decide crear una nueva categoría de trámite, el equipo de desarrollo únicamente deberá crear una nueva clase concreta que implemente **SlaStrategy** y agregar la condición mínima en el **Resolver**, sin riesgo de introducir errores en los trámites ya existentes.

5. Conclusiones

5.1. Impacto de los patrones en la mantenibilidad y testabilidad del sistema

La implementación de los patrones de diseño (Factory, Facade y Strategy) ha transformado sustancialmente la arquitectura del Módulo de Aprobaciones de la plataforma Campus360. Lejos de ser un mero ejercicio académico, esta refactorización generó un impacto directo y medible en dos atributos de calidad fundamentales: la mantenibilidad y la testabilidad.

Impacto en la Mantenibilidad: Desde la perspectiva del mantenimiento del código, el sistema ha alcanzado un alto nivel de cohesión y un bajo acoplamiento. Al respetar estrictamente los principios SOLID —específicamente el Principio de Responsabilidad Única (SRP) y el Principio Abierto/Cerrado (OCP)—, el código fuente es ahora altamente predecible y fácil de leer. Las futuras modificaciones normativas de la universidad (como la adición de nuevos tipos de trámites o la alteración de los Acuerdos de Nivel de Servicio) se podrán integrar extendiendo el sistema mediante nuevas clases (por ejemplo, nuevas estrategias concretas), minimizando el riesgo de introducir regresiones o bugs en el código de producción ya estabilizado.

Impacto en la Testabilidad (Aislamiento y Mocks): En cuanto a la testabilidad, el impacto ha sido determinante para cumplir con los estándares de calidad y cobertura exigidos por el proyecto. En el diseño inicial, la lógica de negocio estaba fuertemente acoplada a las consultas de la base de datos, lo que hacía imposible probar el código sin una conexión activa a MySQL.

Gracias al Principio de Inversión de Dependencias (DIP) inherente a patrones como el Factory y el Strategy, se logró aislar el "Core" del dominio. Esto facilitó enormemente la creación y aplicación de Mocks y Stubs en nuestras pruebas unitarias. Por ejemplo, al validar el comportamiento de la **BandejaAprobacionFacade**, ahora podemos inyectar un repositorio falso (mockeado) utilizando la librería **unittest.mock**. Esto permite simular escenarios complejos (bandejas vacías, SLA vencidos, fallos de lectura) en milisegundos y sin tocar la base de datos real. El resultado es un conjunto de pruebas unitarias robustas, deterministas y aisladas, garantizando la estabilidad del Módulo de Aprobaciones ante cualquier refactorización futura.